

AER1513 STATE ESTIMATION

Assignment 2 Report

"LOST IN THE WOODS DATASET"

Qilong (Jerry) Cheng 1003834103

February 24, 2026

1 Question 1

Based on the data above, is the assumption of zero-mean Gaussian noise reasonable? What values of the variances, Q_k and R_{lk} , should we use?

The noise variables are defined as follows:

$$\begin{aligned} w_k &\sim \mathcal{N}(0, Q_k), \\ n_k^l &\sim \mathcal{N}(0, R_k^l). \end{aligned}$$

Answer:

Given the nonlinear motion model:

$$\begin{bmatrix} x_k \\ y_k \\ \theta_k \end{bmatrix} = \begin{bmatrix} x_{k-1} \\ y_{k-1} \\ \theta_{k-1} \end{bmatrix} + T \begin{bmatrix} \cos(\theta_{k-1}) & 0 \\ \sin(\theta_{k-1}) & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} v_k \\ \omega_k \end{bmatrix} + w_k$$

where T is the sampling time, v_k and ω_k are the translational and rotational speeds of the robot, and $w_k = [w_{v,k}, w_{\omega,k}]$ represents the process noise which is assumed to be zero mean Gaussian white noise with covariance matrix Q_k .

Given the nonlinear observation model:

$$\begin{bmatrix} r_k^l \\ \phi_k^l \end{bmatrix} = \begin{bmatrix} \sqrt{(x_l - x_k - d \cos(\theta_k))^2 + (y_l - y_k - d \sin(\theta_k))^2} \\ \text{atan2}(y_l - y_k - d \sin(\theta_k), x_l - x_k - d \cos(\theta_k)) - \theta_k \end{bmatrix} + n_k^l$$

where $n_k^l = [n_{r,k}^l, n_{\phi,k}^l]$ represents the observation noise which is assumed to be zero mean Gaussian white noise with covariance matrix R_k^l .

1.1 Process Noise Covariance Q_k

The process noise covariance matrix Q_k is associated with the uncertainties in the robot's motion. Specifically, these uncertainties are related to the translational speed v and the rotational speed ω of the robot.

$$Q_k = \begin{bmatrix} \sigma_v^2 & 0 \\ 0 & \sigma_\omega^2 \end{bmatrix} = \begin{bmatrix} v_{var} & 0 \\ 0 & \omega_{var} \end{bmatrix} = \begin{bmatrix} 0.0044 & 0 \\ 0 & 0.0082 \end{bmatrix} \quad (1)$$

Where:

- σ_v^2 is the variance of the translational speed. This represents the uncertainty in the robot's forward motion and is given by the variable `v var` in the dataset.
- σ_ω^2 is the variance of the rotational speed. This represents the uncertainty in the robot's rotational motion and is given by the variable `om var` in the dataset.

1.2 Measurement Noise Covariance R_k^l

The measurement noise covariance matrix R_k^l is associated with the uncertainties in the laser rangefinder measurements. Specifically, these uncertainties are related to the range r and the bearing ϕ to the landmarks.

$$R_k^l = \begin{bmatrix} \sigma_r^2 & 0 \\ 0 & \sigma_\phi^2 \end{bmatrix} = \begin{bmatrix} r_{var} & 0 \\ 0 & b_{var} \end{bmatrix} = \begin{bmatrix} 9.0036 \times 10^{-4} & 0 \\ 0 & 6.7143 \times 10^{-4} \end{bmatrix} \quad (2)$$

Where:

- σ_r^2 is the variance of the range measurement. This represents the uncertainty in the distance measurement from the robot to a landmark and is given by the variable `r var` in the dataset.

- σ_ϕ^2 is the variance of the bearing measurement. This represents the uncertainty in the angular measurement from the robot to a landmark and is given by the variable `b_var` in the dataset.

where v_{var} , om_{var} are the variances of the process noise affecting the robot's transnational and rotational odometry measurement inputs states, and r_{var} , b_{var} are the variances of the range and bearing readings, respectively.

2 Question 2

Write out expressions for all the Jacobians (of the motion and observation models) that are required in the extended Kalman filter.

Answer:

The Extended Kalman Filter (EKF) linearizes the nonlinear motion and observation models about the current estimate using the first-order Taylor expansion, requiring the computation of the Jacobians.

2.1 Jacobian of the Motion Model

Given the nonlinear motion model:

$$\begin{bmatrix} x_k \\ y_k \\ \theta_k \end{bmatrix} = \begin{bmatrix} x_{k-1} \\ y_{k-1} \\ \theta_{k-1} \end{bmatrix} + T \begin{bmatrix} \cos(\theta_{k-1}) & 0 \\ \sin(\theta_{k-1}) & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} v_k \\ \omega_k \end{bmatrix} + T \begin{bmatrix} \cos(\theta_{k-1}) & 0 \\ \sin(\theta_{k-1}) & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} w_{k1} \\ w_{k2} \end{bmatrix}$$

The process noise and is distributed as $w_k \sim \mathcal{N}(0, Q_k)$.

To linearize this model, we compute the Jacobian of f with respect to w_k evaluated at $\hat{x}_{k-1}, v_k, 0$. We derive the Jacobian matrix with respect to the state vector $[x, y, \theta]^T$:

$$\frac{\partial f}{\partial x_k} = \begin{bmatrix} \frac{\partial x_k}{\partial x_k} & \frac{\partial x_k}{\partial y_k} & \frac{\partial x_k}{\partial \theta_k} \\ \frac{\partial y_k}{\partial x_k} & \frac{\partial y_k}{\partial y_k} & \frac{\partial y_k}{\partial \theta_k} \\ \frac{\partial \theta_k}{\partial x_k} & \frac{\partial \theta_k}{\partial y_k} & \frac{\partial \theta_k}{\partial \theta_k} \end{bmatrix}$$

Computing each term:

For x_k :

$$\begin{aligned} \frac{\partial x_k}{\partial x_k} &= 1 \\ \frac{\partial x_k}{\partial y_k} &= 0 \\ \frac{\partial x_k}{\partial \theta_k} &= -T v_k \sin(\theta_{k-1}) \end{aligned}$$

For y_k :

$$\begin{aligned} \frac{\partial y_k}{\partial x_k} &= 0 \\ \frac{\partial y_k}{\partial y_k} &= 1 \\ \frac{\partial y_k}{\partial \theta_k} &= T v_k \cos(\theta_{k-1}) \end{aligned}$$

For θ_k :

$$\begin{aligned}\frac{\partial \theta_k}{\partial x_k} &= 0 \\ \frac{\partial \theta_k}{\partial y_k} &= 0 \\ \frac{\partial \theta_k}{\partial \theta_k} &= 1\end{aligned}$$

Combining these results, the Jacobian matrix is:

$$\frac{\partial f}{\partial x_k} = \begin{bmatrix} 1 & 0 & -Tv_k \sin(\theta_{k-1}) \\ 0 & 1 & Tv_k \cos(\theta_{k-1}) \\ 0 & 0 & 1 \end{bmatrix}$$

From the Jacobian of the noise term, we can derive that:

$$\frac{\partial f}{\partial w_k} = \begin{bmatrix} \frac{\partial x_k}{\partial w_{k1}} & \frac{\partial x_k}{\partial w_{k2}} \\ \frac{\partial y_k}{\partial w_{k1}} & \frac{\partial y_k}{\partial w_{k2}} \\ \frac{\partial \theta_k}{\partial w_{k1}} & \frac{\partial \theta_k}{\partial w_{k2}} \end{bmatrix}$$

For w_{k1}

$$\begin{aligned}\frac{\partial x_k}{\partial w_{k1}} &= Tv_k \cos(\theta_{k-1}) \\ \frac{\partial x_k}{\partial w_{k2}} &= 0 \\ \frac{\partial y_k}{\partial w_{k1}} &= Tv_k \sin(\theta_{k-1})\end{aligned}$$

For w_{k2}

$$\begin{aligned}\frac{\partial y_k}{\partial w_{k2}} &= 0 \\ \frac{\partial \theta_k}{\partial w_{k1}} &= 0 \\ \frac{\partial \theta_k}{\partial w_{k2}} &= T\end{aligned}$$

Substituting these values in, we get:

$$\frac{\partial f}{\partial w_k} = \begin{bmatrix} Tv_k \cos(\theta_{k-1}) & 0 \\ Tv_k \sin(\theta_{k-1}) & 0 \\ 0 & T \end{bmatrix}$$

This Jacobian matrix is denoted as W .

Now, w'_k is given by:

$$w'_k = Ww_k$$

Given the Jacobian matrix W and the noise covariance matrix Q_k , the expression for Q'_k is:

$$Q'_k = WQ_kW^T$$

Multiplying out the matrices, we get:

$$Q'_k = \begin{bmatrix} T^2 v_{var} v_k^2 \cos^2(\theta_{k-1}) & T^2 v_{var} v_k^2 \cos(\theta_{k-1}) \sin(\theta_{k-1}) & 0 \\ T^2 v_{var} v_k^2 \cos(\theta_{k-1}) \sin(\theta_{k-1}) & T^2 v_{var} v_k^2 \sin^2(\theta_{k-1}) & 0 \\ 0 & 0 & T^2 \sigma_{var} \end{bmatrix}$$

This is the expression for Q'_k in matrix form.

Where Q_k is the covariance matrix of w_k .

2.2 Jacobian of the Observation Model

The observation model $z_k = h(x_k) + n_k$ relates the state x_k to the observed range and bearing (r_{kl}, ϕ_k^l) through the nonlinear function h :

$$\begin{bmatrix} r_k^l \\ \phi_k^l \end{bmatrix} = \begin{bmatrix} \sqrt{(x_l - x_k - d \cos(\theta_k))^2 + (y_l - y_k - d \sin(\theta_k))^2} \\ \text{atan2}(y_l - y_k - d \sin(\theta_k), x_l - x_k - d \cos(\theta_k)) - \theta_k \end{bmatrix} + \begin{bmatrix} n_r \\ n_\phi \end{bmatrix}$$

Derivation for r_k^l :

1. With respect to x_k :

$$\frac{\partial r_k^l}{\partial x_k} = \frac{x_k - x_l + d \cos(\theta_k)}{\sqrt{(x_l - x_k - d \cos(\theta_k))^2 + (y_l - y_k - d \sin(\theta_k))^2}}$$

2. With respect to y_k :

$$\frac{\partial r_k^l}{\partial y_k} = \frac{y_k - y_l + d \sin(\theta_k)}{\sqrt{(x_l - x_k - d \cos(\theta_k))^2 + (y_l - y_k - d \sin(\theta_k))^2}}$$

3. With respect to θ_k :

$$\frac{\partial r_k^l}{\partial \theta_k} = \frac{(x_k - x_l + d \cos(\theta_k))d \sin(\theta_k) + (y_k - y_l + d \sin(\theta_k))d \cos(\theta_k)}{\sqrt{(x_l - x_k - d \cos(\theta_k))^2 + (y_l - y_k - d \sin(\theta_k))^2}}$$

Derivation for ϕ_k^l :

1. With respect to x_k :

$$\frac{\partial \phi_k^l}{\partial x_k} = \frac{y_l - y_k - d \sin(\theta_k)}{(x_l - x_k - d \cos(\theta_k))^2 + (y_l - y_k - d \sin(\theta_k))^2}$$

2. With respect to y_k :

$$\frac{\partial \phi_k^l}{\partial y_k} = \frac{-x_l + x_k + d \cos(\theta_k)}{(x_l - x_k - d \cos(\theta_k))^2 + (y_l - y_k - d \sin(\theta_k))^2}$$

3. With respect to θ_k :

$$\frac{\partial \phi_k^l}{\partial \theta_k} = -1 - \frac{d(y_l - y_k) \cos(\theta_k) - d(x_l - x_k) \sin(\theta_k)}{(x_l - x_k - d \cos(\theta_k))^2 + (y_l - y_k - d \sin(\theta_k))^2}$$

Jacobian Matrix G_k :

Combining the above partial derivatives, we get the Jacobian matrix G_k :

$$G_k = \begin{bmatrix} \frac{\partial r_k^l}{\partial x_k} & \frac{\partial r_k^l}{\partial y_k} & \frac{\partial r_k^l}{\partial \theta_k} \\ \frac{\partial \phi_k^l}{\partial x_k} & \frac{\partial \phi_k^l}{\partial y_k} & \frac{\partial \phi_k^l}{\partial \theta_k} \end{bmatrix}$$

The noise term n_k is added directly to the observation, so the Jacobian of the observation model with respect to the noise term is an identity matrix:

$$H = \frac{\partial h(x_k)}{\partial n_k} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Given that $n'_k = H n_k$, we have:

$$n'_k = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} n_r \\ n_\phi \end{bmatrix} = \begin{bmatrix} n_r \\ n_\phi \end{bmatrix}$$

The covariance of n'_k is given by:

$$R'_k = H R_k^l H^T$$

Given:

$$R_k^l = \begin{bmatrix} r_{\text{var}} & 0 \\ 0 & b_{\text{var}} \end{bmatrix}$$

Substituting in the values, we get:

$$R'_k = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} r_{\text{var}} & 0 \\ 0 & b_{\text{var}} \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} r_{\text{var}} & 0 \\ 0 & b_{\text{var}} \end{bmatrix}$$

This is the expression for R'_k in matrix form.

3 Question 3

Modify and write out the equations of the extended Kalman filter, modified to handle a variable number of exteroceptive measurements at each timestep.

Answer:

When there's a variable number of exteroceptive measurements at each timestep, the EKF's update step needs to be executed for each available measurement. This is because each measurement provides a unique observation about the state of the system, and the EKF should incorporate all available information to refine its state estimate.

3.1 Prediction Step

Predict the state:

$$\check{x}_k = f(\hat{x}_{k-1}, v_k, 0) \quad (3)$$

Predict the covariance:

$$\check{P}_k = F_{k-1} \hat{P}_{k-1} F_k^T + Q'_k \quad (4)$$

3.2 Update Step for Each Exteroceptive Measurement

For each available exteroceptive measurement l at timestep k : Compute the Kalman gain for the l^{th} measurement:

$$K_k^l = \check{P}_k G_k^{lT} (G_k^l \check{P}_k G_k^{lT} + R_k^l)^{-1} \quad (5)$$

Update the state estimate using the l^{th} measurement:

$$\hat{x}_k = \check{x}_k + K_k^l (y_k^l - g(\check{x}_k, l)) \quad (6)$$

Update the covariance estimate:

$$\hat{P}_k = (I - K_k^l H_k^l) \check{P}_k \quad (7)$$

In this step, we can either loop through each valid measurement individually. (Valid measurement requires to be non-zero and less than the r_{max} value)

3.3 Jacobian Matrices F and G

Given the state transition function:

$$f(x_{k-1}, v_k, w_k)$$

and the measurement function:

$$g(x_k, n_k)$$

The Jacobian matrices F and G are derived as:

$$F_{k-1} = \left. \frac{\partial f(x_{k-1}, v_k, w_k)}{\partial x_{k-1}} \right|_{x_{k-1}=\hat{x}_{k-1}, v_k, w_k=0} \quad (8)$$

$$G_k^l = \left. \frac{\partial g(x_k, n_k)}{\partial x_k} \right|_{x_k=\check{x}_k, n_k=0} \quad (9)$$

3.4 Modified Covariance Q'

From the provided notation, the modified process noise is given by:

$$w' = \left. \frac{\partial f(x_{k-1}, v_k, w_k)}{\partial w_k} \right|_{x_{k-1}=\hat{x}_{k-1}, v_k, w_k=0}$$

Given the noise distributions:

$$w \sim N(0, Q)$$

$$w' \sim N(0, Q')$$

The modified covariance Q' is derived as:

$$Q' = w' Q w'^T \quad (10)$$

The modification of the EKF to this problem is that in the measurement model, the size of the matrix is dependent on the number of valid measurements there are. Before going through the last three steps on the EKF algorithm, the range value is first checked if it is non-zero and smaller than the required r_{max} . If both conditions satisfy, the posterior for x and P processed to update.

4 Question 4

Write a MATLAB or Python script to compute an estimate of the robot poses, $\hat{x}_k = (x_k, y_k, \theta_k)$, using the extended Kalman filter. Your code should be able to artificially limit itself to only use range/bearing landmark observations for which the range is less than a user-defined threshold, r_{max} [m]. Note, the true robot position, $x_k = (x_k, y_k, \theta_k)$, is contained in the x_{true} , y_{true} , and θ_{true} arrays.

Instructions and Requirements

1. Use $\hat{x}_0 = x_0$, $\hat{P}_0 = \text{diag}\{1, 1, 0.1\}$ as the initial condition (i.e., the estimator mean starts with the right answer). Plot the following quantities on the same axes:
 - The error, $\hat{x}_k - x_k$ vs. t_k , as a solid line.
 - The uncertainty envelope, $\pm 3\sigma_x$ vs. t_k , as two dotted lines. Note, the variance, σ_x^2 , can be extracted from the diagonal of the covariance matrix, $\hat{P}_k$.

Make one plot for each value of $r_{max} = 5, 3, 1$. Repeat for y and θ . That's 9 plots in total.

Answer:

For the Extended Kalman Filter derivation, we have iterated the last three steps for each valid measurement. The experiments were conducted with maximum range values for the measurements set to $r_{max} = 5, 3$, and 1 , respectively. The results from the plots can be summarized as follows:

- The error, denoted as $\hat{x}_k - x_k$, is plotted against t_k as a solid line.
- The uncertainty envelope, represented by $\pm 3\sigma_x$, is plotted against t_k as two dotted lines. The variance, σ_x^2 , is extracted from the diagonal of the covariance matrix, $\hat{P}_k$.
- A total of nine plots were generated, three for each state variable (x , y , and θ), with each plot corresponding to one of the r_{max} values.

From the plots, we can infer the following:

- Having a higher number of measurements (i.e., a larger r_{max}) significantly enhances the accuracy of the estimates.
- Even with sparse measurement data, the Extended Kalman Filter is robust enough to adjust and converge swiftly to the ground truth.
- The pose estimate accuracy achieved by incorporating the measurements is substantially better than relying solely on the motion model.

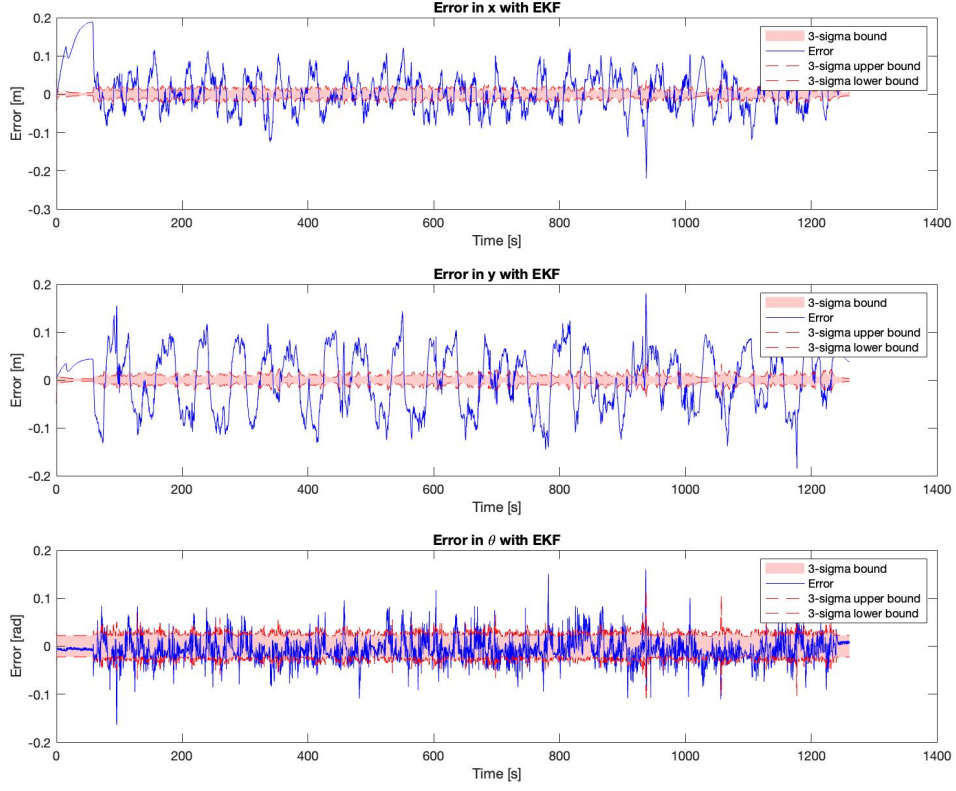


Figure 1: Plot of the estimated pose of the robot given $r_{max} = 5$, with the initial condition given as x_{true}

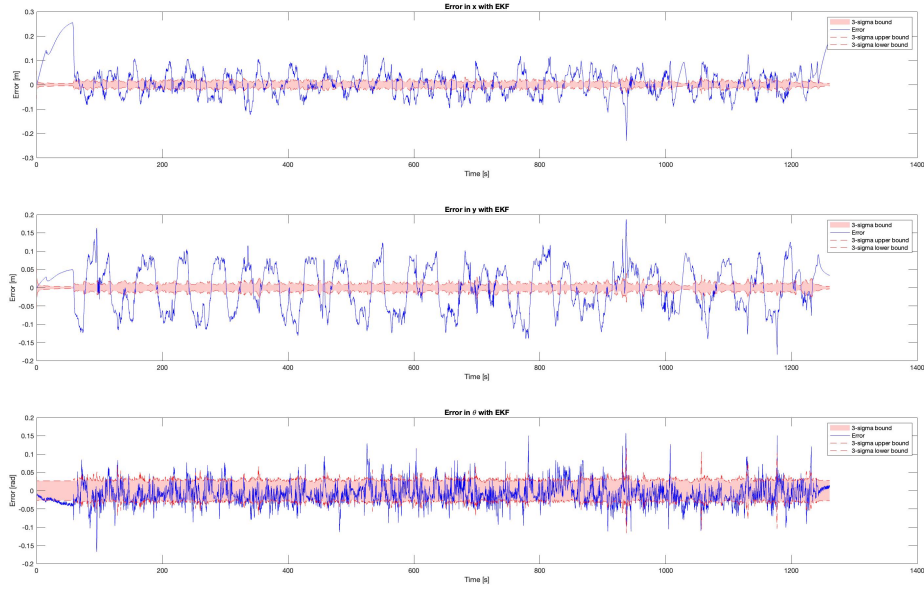


Figure 2: Plot of the estimated pose of the robot given $r_{max} = 3$, with the initial condition given as x_{true}

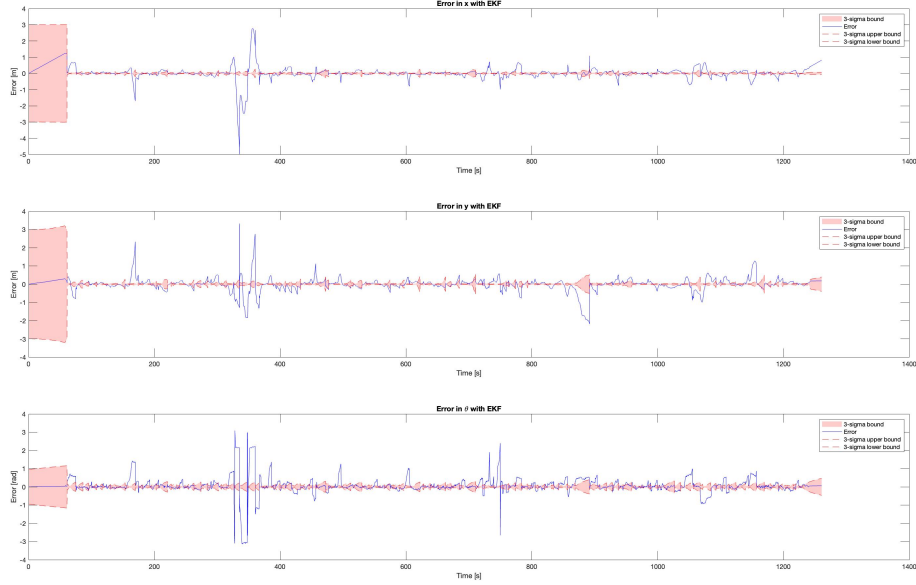


Figure 3: Plot of the estimated pose of the robot given $r_{max} = 1$, with the initial condition given as x_{true}

2. Repeat (a) with $\hat{x}_0 = (1, 1, 0.1)$, i.e., a poor initial condition. See how far you can push the initial condition away and still have your estimator converge.

Answer:

We have conducted experiments with various initial conditions for the estimation problem. Specifically:

- (a) The first experiment utilized an initial condition of $\hat{x}_0 = (1, 1, 0.1)$.
- (b) The second experiment set the initial condition to $\hat{x}_0 = (10, 10, 10)$.
- (c) Lastly, the third experiment pushed the initial condition to an extreme with $\hat{x}_0 = (9999, 9999, 9999)$.

From the results visualized in the plots, we can draw the following conclusions:

- Even when starting with a poorly estimated initial condition, the estimator converges to the correct value in a relatively short number of iterations.
- The rate of convergence is influenced by the number of measurements available. Specifically, with a higher number of measurements, the estimator converges faster compared to scenarios with fewer measurements.

This experiment demonstrates the robustness of the estimator, even when initialized far from the true state.

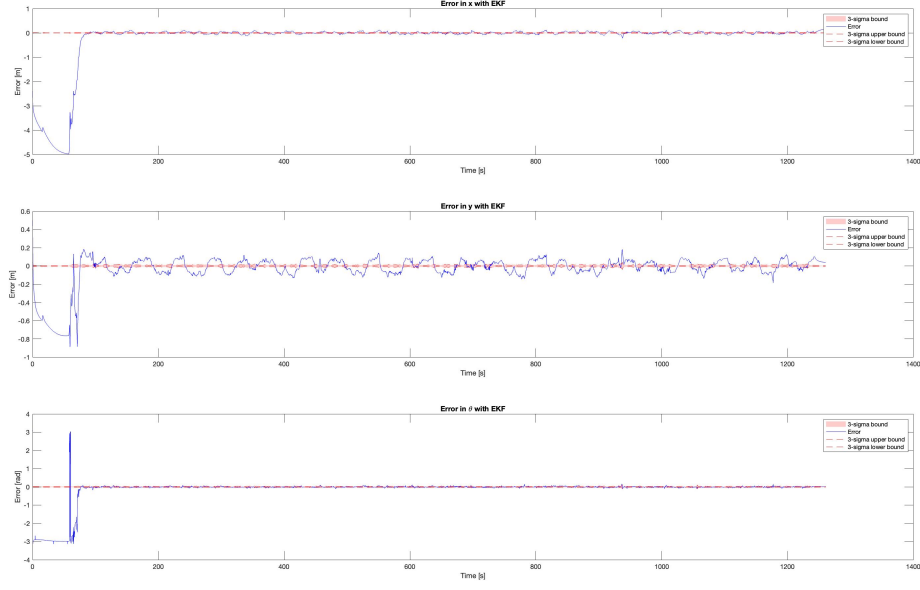


Figure 4: Plot of the estimated pose of the robot given $r_{max} = 5$, with the initial condition given as $x_{init} = [1; 1; 0.1]$

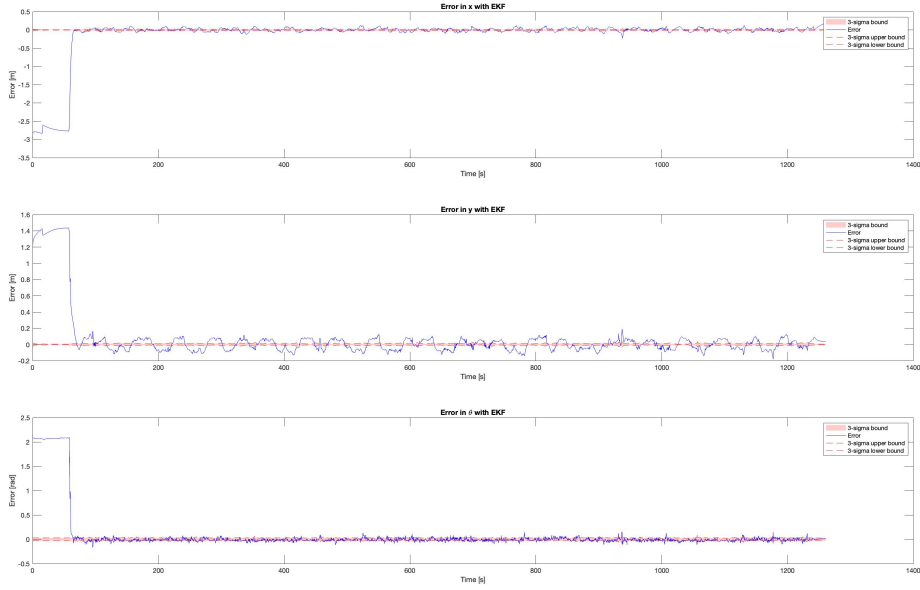


Figure 5: Plot of the estimated pose of the robot given $r_{max} = 3$, with the initial condition given as $x_{init} = [1; 1; 0.1]$

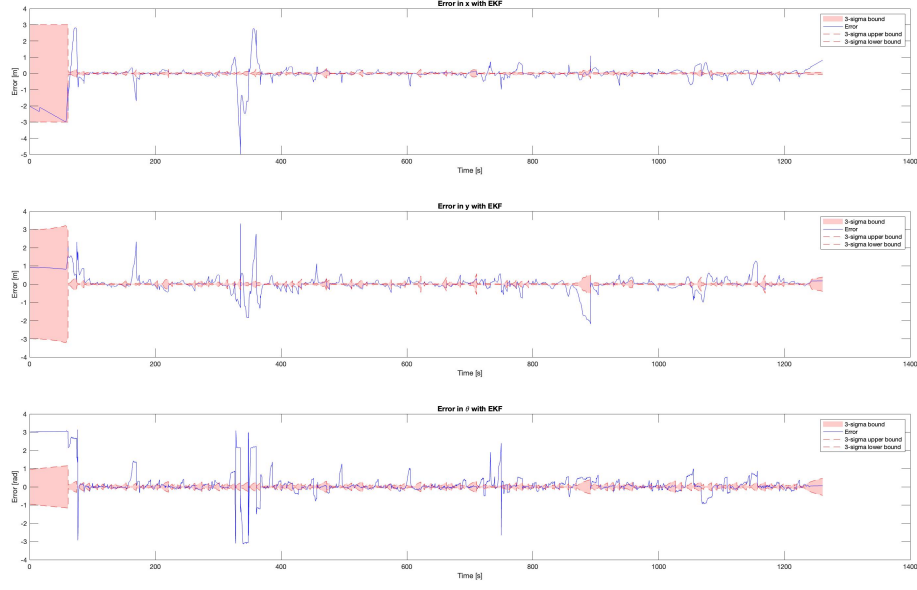


Figure 6: Plot of the estimated pose of the robot given $r_{max} = 1$, with the initial condition given as $x_{init} = [1; 1; 0.1]$

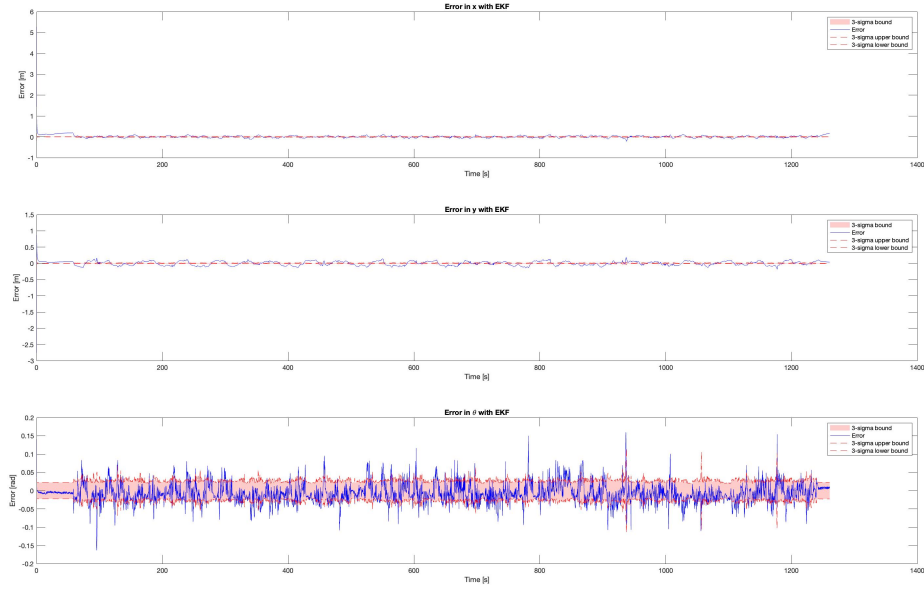


Figure 7: Plot of the estimated pose of the robot given $r_{max} = 5$, with the initial condition given as $x_{init} = [10; 10; 10]$

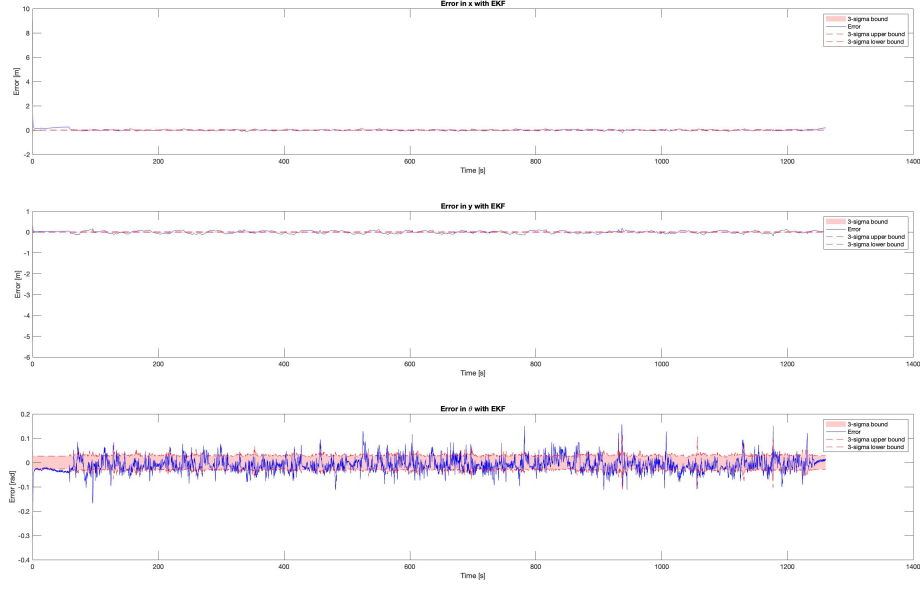


Figure 8: Plot of the estimated pose of the robot given $r_{max} = 3$, with the initial condition given as $x_{init} = [10; 10; 10]$

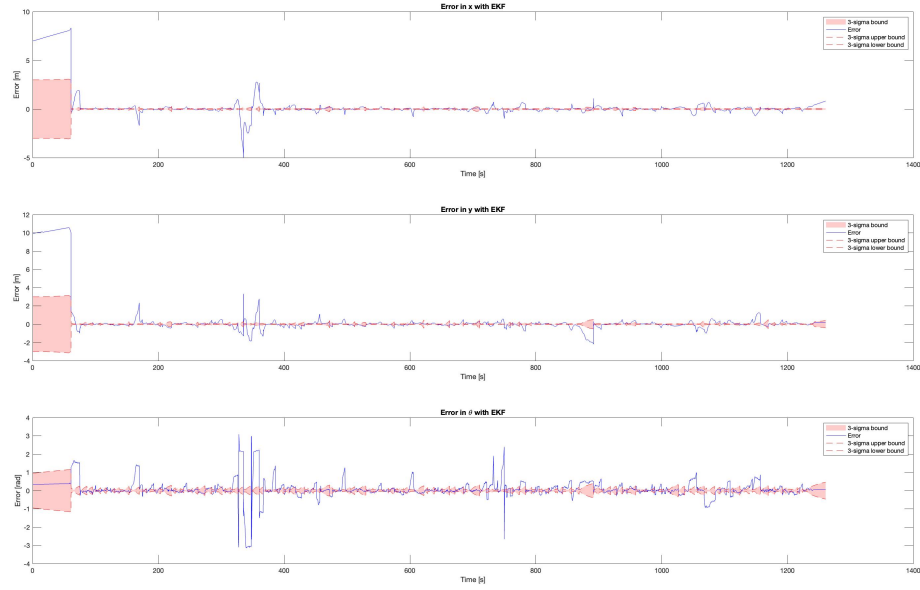


Figure 9: Plot of the estimated pose of the robot given $r_{max} = 1$, with the initial condition given as $x_{init} = [10; 10; 10]$

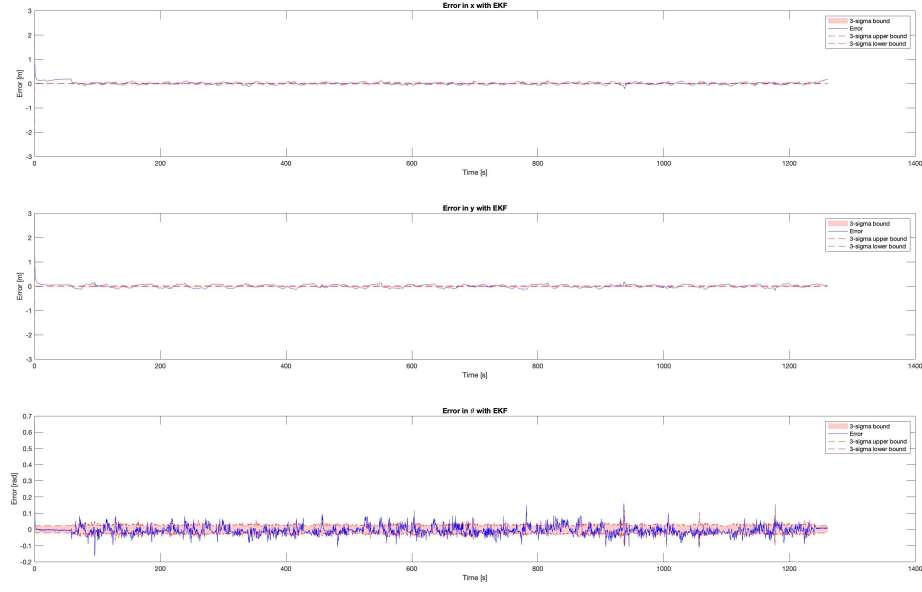


Figure 10: Plot of the estimated pose of the robot given $r_{max} = 5$, with the initial condition given as $x_{init} = [9999; 9999; 9999]$

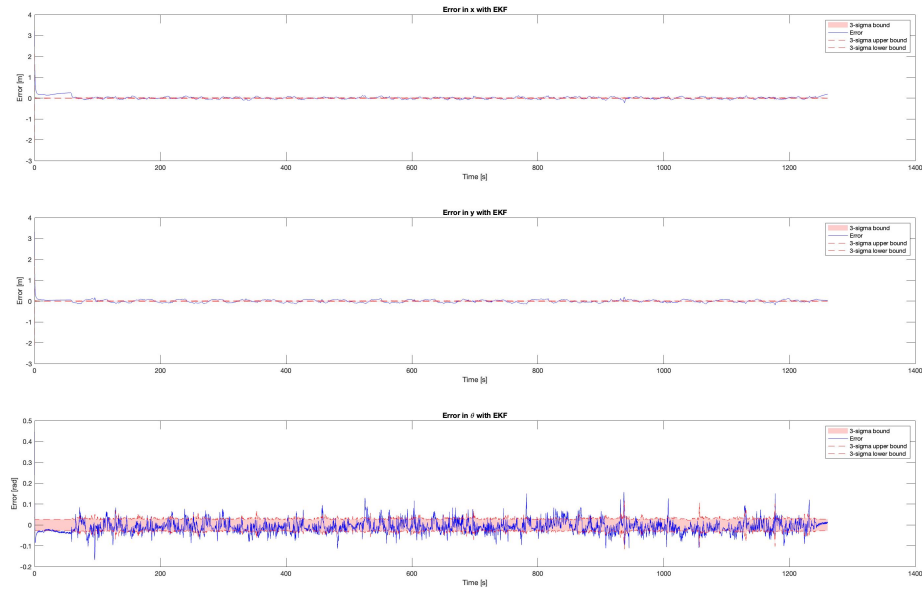


Figure 11: Plot of the estimated pose of the robot given $r_{max} = 3$, with the initial condition given as $x_{init} = [9999; 9999; 9999]$

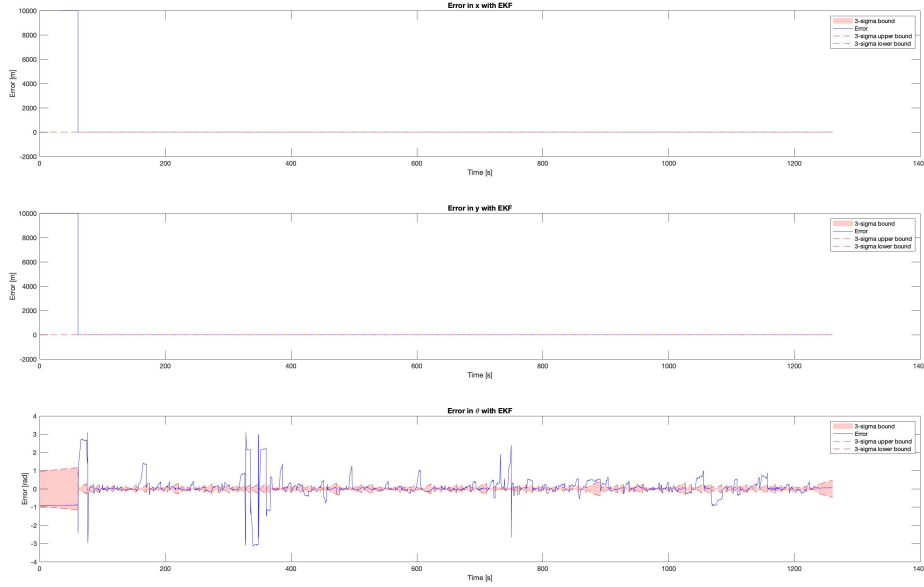


Figure 12: Plot of the estimated pose of the robot given $r_{max} = 1$, with the initial condition given as $x_{init} = [9999; 9999; 9999]$

3. Repeat (a) where you evaluate all the Jacobians at the true robot state, x_k , instead of the estimated robot state, $\hat{x}_k$. This is the CRLB version of the EKF. Contrast this to (a).

Answer:

The Extended Kalman Filter (EKF) linearizes nonlinear system dynamics using Jacobians. The point of linearization significantly influences the EKF's performance.

(a) Evaluation Using Estimated State, $\hat{x}_k$:

- Assumes the current estimate is close to the true state.
- Errors in the estimate can propagate, affecting filter performance.

(b) Evaluation Using Ground Truth, x_k :

- Provides "perfect" linearization, representing the EKF's best possible performance (Cramér-Rao Lower Bound, CRLB).
- Avoids errors introduced by imperfect linearization.

The primary difference between the two approaches is the point of linearization. While the standard EKF uses its best guess (the estimated state), the CRLB version uses the actual state. As a result, the CRLB version of the EKF often provides more accurate state estimates, especially in scenarios where the system's dynamics or measurements are highly nonlinear. However, in real-world applications, the true state is typically unknown, making the standard EKF approach more practical. The CRLB version serves as a benchmark, indicating the best possible performance under ideal conditions.

In conclusion, using the ground truth robot state for Jacobian evaluation in the EKF provides insights into the filter's optimal performance. It showcases the importance of accurate linearization in state estimation and highlights potential areas for improvement in the standard EKF approach. While the CRLB version offers superior performance, it's essential to recognize its theoretical nature and the challenges of achieving such performance in real-world scenarios.

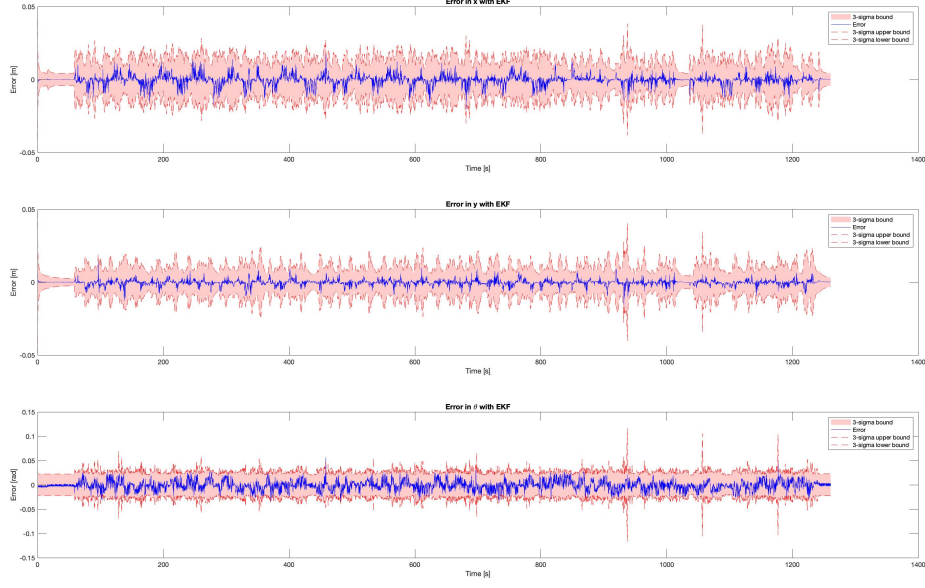


Figure 13: Plot of the estimated pose of the robot given $r_{max} = 5$, with the initial condition given as $x_{init} = [1; 1; 0.1]$, and the Jacobians evaluated at x_{true} using CRLB method

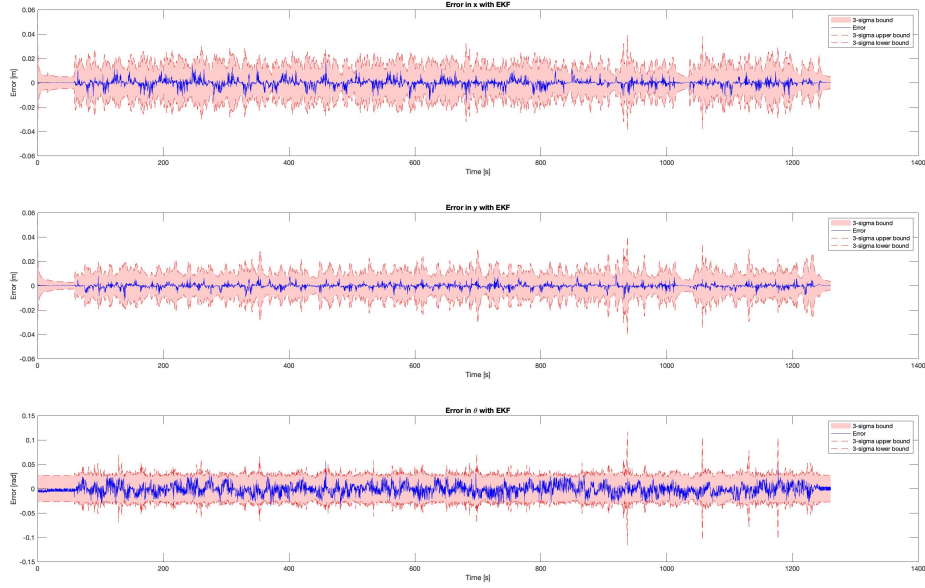


Figure 14: Plot of the estimated pose of the robot given $r_{max} = 3$, with the initial condition given as $x_{init} = [1; 1; 0.1]$, and the Jacobians evaluated at x_{true} using CRLB method

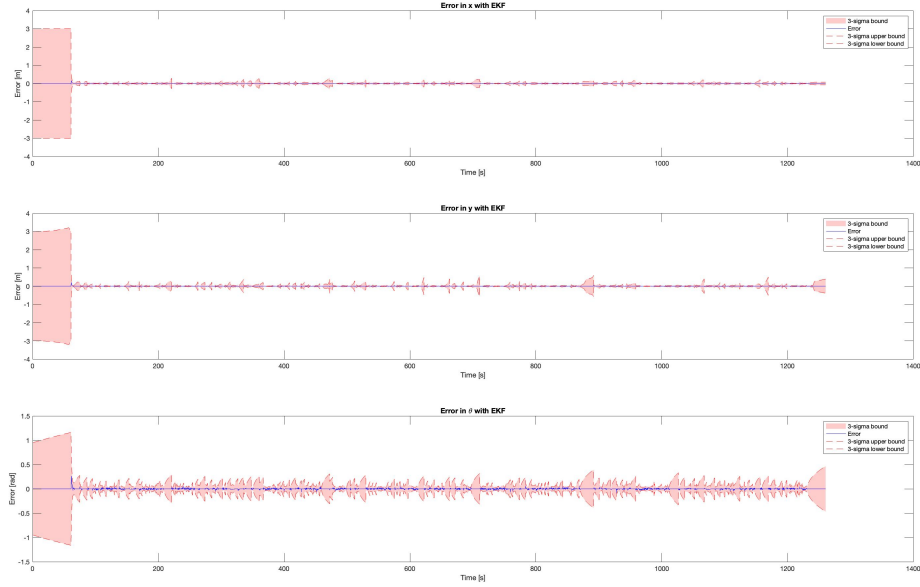


Figure 15: Plot of the estimated pose of the robot given $r_{max} = 1$, with the initial condition given as $x_{init} = [1; 1; 0.1]$, and the Jacobians evaluated at x_{true} using CRLB method

5 Question 5

Create an animation of your EKF estimate for the case $r_{max} = 1$ and $\hat{x}_0 = x_0$, $\hat{P}_0 = \text{diag}\{1, 1, 0.1\}$. The animation should clearly display the following information:

- True landmarks positions, represented by static dots (black).
- True robot position, (x_k, y_k) , depicted as a moving dot (blue).
- Estimated robot position, $(\hat{x}_k, \hat{y}_k)$, depicted as a moving dot (red).
- 3-sigma covariance ellipse, using the top-left 2×2 submatrix of $\hat{P}_k$. This ellipse should be centered on the estimated robot position. It is desired to observe that the true robot position stays within this ellipse, although it might not always do so. Represent this with a moving ellipse (red).

The completed animation should be submitted as a movie file compatible with standard media players, such as VLC. Ensure the content is clearly visible and the animation reflects the real-time estimation process of the EKF.

Answer:

Check for the attached video in the zip file.

Appendix - MATLAB Source Code

```
1 clear all;
2 clc;
3
4 %% Load the dataset
5 load dataset2.mat;
6 who;
7
8 %% Estimating for robot pose x
9 % Initial conditions
10 x0_est = [x_true(1); y_true(1); th_true(1)]; % a good estimate of the robot pose
11 % x0_est = [1; 1; 0.1];
12 P0 = diag([1, 1, 0.1]);
13
14 % Placeholder for EKF results
15 x_posterior = zeros(3, length(t));
16 P_posterior = zeros(3, 3, length(t));
17
18 x_open_loop = zeros(3, length(t));
19
20
21 % EKF initialization
22 x_posterior(:, 1) = x0_est;
23 P_posterior(:, :, 1) = P0;
24
25 landmarks = 1; % After loading the dataset
26
27 % Define constants
28 T = 0.1; % Assuming a sampling period of 0.1s
29
30 % Process noise covariance matrix
31 Qk = [v_var, 0;
32       0, om_var];
33
34 % Measurement noise covariance matrix
35 Rk = [r_var, 0;
36       0, b_var];
37
38 rmax = 5; % or any other value you want to set as the threshold
39
40 pose_true = [x_true, y_true, th_true]';
41 x_open_loop(:, 1) = pose_true(:, 1);
42
43 CRLB = false;
44
45 theta_open_loop = 0;
46 for k = 2:length(t)
47     x_open_loop(1, k) = x_open_loop(1, k-1) + T * cos(theta_open_loop)*v(k);
48     x_open_loop(2, k) = x_open_loop(2, k-1) + T * sin(theta_open_loop)*v(k);
49     x_open_loop(3, k) = wrapToPi(x_open_loop(3, k-1) + T * om(k));
50     theta_open_loop = x_open_loop(3, k);
51
52 end
53
54 %% EKF with stack
55
56 for k = 2:length(t)
57     % Initialize measurement matrix and Jacobian for each time step
58     Z_pred = [];
59     Z_actual = [];
60     Z_error = [];
```

```

61
62 Gk_stack = [];
63 Rk_stack = [];
64 update_status = false;
65
66 % Prediction step
67 if CRLB == true
68     theta_posterior = th_true(k-1);
69 else
70     theta_posterior = wrapToPi(x_posterior(3, k-1));
71 end
72
73 Fk = [1, 0, T*v(k)*sin(theta_posterior);
74       0, 1, T*v(k)*cos(theta_posterior);
75       0, 0, 1];
76
77 % Update Qk matrix
78 Qk = [T^2*v_var * v(k)^2 * cos(theta_posterior)^2,
79       T^2*v_var * v(k)^2 * cos(theta_posterior) * sin(theta_posterior),
80       0;
81       T^2*v_var * v(k)^2 * cos(theta_posterior) * sin(theta_posterior),
82       T^2*v_var * v(k)^2 * sin(theta_posterior)^2,
83       0;
84       0,
85       0,
86       T^2*om_var];
87
88 %% Predictors:
89 % Calculate x prior
90 x_prior = x_posterior(:, k-1) + T * [cos(theta_posterior)*v(k); sin(
    theta_posterior)*v(k); om(k)];
91
92 % calculate x when it is only based on the motion model:
93 x_open_loop(1, k) = x_open_loop(1, k-1) + T * cos(x_open_loop(3, k))*v(k);
94 x_open_loop(2, k) = x_open_loop(2, k-1) + T * sin(x_open_loop(3, k))*v(k);
95 x_open_loop(3, k) = x_open_loop(3, k-1) + T * om(k);
96 x_open_loop(3, k) = wrapTo2Pi(x_open_loop(3, k));
97
98 % Calculate P prior
99 P_prior = Fk * P_posterior(:, :, k-1) * Fk' + Qk;
100
101 % Update the theta before calculating the measurement matrix
102 if CRLB == true
103     theta_prior = th_true(k);
104     x_prior = pose_true(:, k);
105 else
106     theta_prior = wrapToPi(x_prior(3)); % make sure that the prior theta is
        bounded b/w -pi and pi
107 end
108
109 for l = 1:size(r, 2)
110     if r(k, l) < rmax && r(k, l) ~= 0
111         update_status = true;
112
113         % Calculate the position of the laser range finder
114         x_lrf = x_prior(1) + d * cos(theta_prior);
115         y_lrf = x_prior(2) + d * sin(theta_prior);
116
117         % Calculate the difference between the landmark and the LRF
118         dx = landmarks(l, 1) - x_lrf;
119         dy = landmarks(l, 2) - y_lrf;
120         d_lrf = sqrt(dx^2 + dy^2);

```

```

121
122     % Jacobian matrix Gk
123     Gk = [ (x_lrf - landmarks(1, 1))/d_lrf,
124            (y_lrf - landmarks(1, 2))/d_lrf,
125            (d*sin(theta_prior)*(x_lrf - landmarks(1, 1)) ...
126             - d*cos(theta_prior)*(y_lrf - landmarks(1, 2)))/d_lrf^2;
127            (landmarks(1, 2) - y_lrf)/d_lrf^2,
128            -(landmarks(1, 1) - x_lrf)/d_lrf^2,
129            -1 - (d*cos(theta_prior)*(landmarks(1, 2) - y_lrf) ...
130             + d*sin(theta_prior)*(landmarks(1, 1) - x_lrf))/d_lrf^2];
131
132     % Update Rk matrix
133     Rk = [r_var,    0;
134           0,        b_var];
135
136     % Predicted and actual measurements
137     z_pred = [d_lrf; wrapToPi(atan2(dy, dx) - theta_prior)];
138     z_actual = [r(k, 1); b(k, 1)];
139     z_error = wrapToPi(z_actual - z_pred);
140
141     % Stack the measurements and Jacobians
142     Z_pred = [Z_pred; z_pred];
143     Z_actual = [Z_actual; z_actual];
144     %       Z_error = [Z_error; z_error];
145
146     Gk_stack = [Gk_stack; Gk];
147     Rk_stack = blkdiag(Rk_stack, Rk);
148 end
149 end
150
151 if update_status == true
152     disp(['Valid Timesetp Found']);
153     % Kalman Gain:
154     Kk = P_prior * Gk_stack' * inv(Gk_stack * P_prior * Gk_stack' + Rk_stack);
155
156     % Corrector:
157     z_error_old = Z_actual - Z_pred;
158     %       z_error(2:2:end) = z_error(2:2:end); % Adjust the sign of the error term
159     for bearing_angles
160         x_pred = x_prior + Kk * z_error_old;
161         P_pred = (eye(3) - Kk * Gk_stack) * P_prior;
162
163         % Ensure theta is between -pi and pi
164         x_pred(3) = wrapToPi(x_pred(3));
165
166         x_posterior(:, k) = x_pred;
167         P_posterior(:, :, k) = P_pred;
168
169     else
170         x_prior(3) = wrapToPi(x_prior(3));
171         x_posterior(:, k) = x_prior;
172         P_posterior(:, :, k) = P_prior;
173     end
174
175     disp(['The timestep t is: ' num2str(k)]);
176 end
177
178 %% Extracting the estimated states and their uncertainties
179 x_x_est = x_posterior(1, 2:end);
180 P_x_est = squeeze(P_posterior(1, 1, 2:end)); % Extracting the variance for x
181

```

```

182 y_y_est = x_posterior(2, 2:end);
183 P_y_est = squeeze(P_posterior(2, 2, 2:end)); % Extracting the variance for y
184
185 theta_theta_est = x_posterior(3, 2:end);
186 P_theta_est = squeeze(P_posterior(3, 3, 2:end)); % Extracting the variance for theta
187
188 t = t(2:end);
189
190 x_true = x_true(2:end);
191 y_true = y_true(2:end);
192 th_true = th_true(2:end);
193
194 %% Plotting results
195 figure;
196
197 % For x
198 % For x
199 subplot(3,1,1);
200 fill([t; flipud(t)], [3*sqrt(P_x_est); flipud(-3*sqrt(P_x_est))], [1 0.8 0.8], ...
201 'EdgeColor', 'none'); % Shaded region
202 hold on;
203 plot(t, x_x_est'-x_true, 'b', t, 3*sqrt(P_x_est), 'r--', t, -3*sqrt(P_x_est), 'r--');
204 title('Error in x with EKF');
205 xlabel('Time [s]');
206 ylabel('Error [m]');
207 legend('3-sigma bound', 'Error', '3-sigma upper bound', '3-sigma lower bound');
208
209 % For y
210 subplot(3,1,2);
211 fill([t; flipud(t)], [3*sqrt(P_y_est); flipud(-3*sqrt(P_y_est))], [1 0.8 0.8], ...
212 'EdgeColor', 'none'); % Shaded region
213 hold on;
214 plot(t, y_y_est'-y_true, 'b', t, 3*sqrt(P_y_est), 'r--', t, -3*sqrt(P_y_est), 'r--');
215 title('Error in y with EKF');
216 xlabel('Time [s]');
217 ylabel('Error [m]');
218 legend('3-sigma bound', 'Error', '3-sigma upper bound', '3-sigma lower bound');
219
220 % For theta
221 subplot(3,1,3);
222 fill([t; flipud(t)], [3*sqrt(P_theta_est); flipud(-3*sqrt(P_theta_est))], ...
223 [1 0.8 0.8], 'EdgeColor', 'none'); % Shaded region
224 hold on;
225 plot(t, wrapToPi(theta_theta_est'-th_true), 'b', t, 3*sqrt(P_theta_est), 'r--', ...
226 t, -3*sqrt(P_theta_est), 'r--');
227 title('Error in \theta with EKF');
228 xlabel('Time [s]');
229 ylabel('Error [rad]');
230 legend('3-sigma bound', 'Error', '3-sigma upper bound', '3-sigma lower bound');
231
232 %% plot the individual axis for x, y, theta:
233 figure;
234
235 % For x
236 subplot(3,1,1);
237 plot(t, x_true, 'b', t, x_x_est);
238 title('x with EKF');
239 xlabel('Time [s]');
240 ylabel('x [m]');
241 legend('x true', 'x estimate');
242
243

```

```

244 % For y
245 subplot(3,1,2);
246 plot(t, y_true, 'b', t, y_y_est);
247 title('y with EKF');
248 xlabel('Time [s]');
249 ylabel('y [m]');
250 legend('y true', 'y estimate');
251
252
253 % For theta
254 subplot(3,1,3);
255 plot(t, th_true, 'b', t, theta_theta_est);
256 title('\theta with EKF');
257 xlabel('Time [s]');
258 ylabel('theta [rad]');
259 legend('theta true', 'theta estimate');
260
261 %%
262 % Animation
263 v = VideoWriter('lost_in_the_woods_animation_best_result_with_lidar.mp4', 'MPEG-4');
264 v.FrameRate = 10; % Optional: Set the frame rate (default is 30 frames per second)
265 v.Quality = 50; % Optional: Set the quality (0 to 100, higher is better quality)
266 open(v);
267
268 figure;
269 fig = figure;
270 set(fig, 'Position', [100, 100, 1120, 840]); % [left bottom width height]
271 % % Set the axis limits
272 axis equal;
273 xlim([min(x_true) - 1, max(x_true) + 1]);
274 ylim([min(y_true) - 1, max(y_true) + 1]);
275
276 % Plot static landmarks
277 plot(landmarks(:,1), landmarks(:,2), 'k.', 'MarkerSize', 50);
278 hold on;
279
280 % Initialize animated lines for true and estimated paths
281 h_true_path = animatedline('Color', 'b', 'LineWidth', 0.5);
282 h_est_path = animatedline('Color', 'r', 'LineWidth', 0.5);
283 %% Optional
284 % h_open_loop_path = animatedline('Color', 'g', 'LineWidth', 0.5);
285 %%
286 % Plot initial positions
287
288 % Plot static landmarks
289 plot(landmarks(:,1), landmarks(:,2), 'k.', 'MarkerSize', 10);
290 hold on;
291
292 h_true = plot(x_true(1), y_true(1), 'bo', 'MarkerSize', 6, 'LineWidth', 2);
293 h_est = plot(x_x_est(1), y_y_est(1), 'ro', 'MarkerSize', 6, 'LineWidth', 2) %, ...
294 'MarkerFaceColor', 'r'); % with infill added
295 %% Optional
296 % h_open_loop = plot(x_open_loop(1, 1), x_open_loop(2, 1), 'go', ...
297 'MarkerSize', 6, 'LineWidth', 2);
298 %%
299 % Initial 3-sigma covariance ellipse
300 P_2x2 = P_posterior(1:2, 1:2, 1);
301 [V, D] = eig(P_2x2);
302 a_sigma = sqrt(D(1,1))*3; % 3-sigma semi-major axis
303 b_sigma = sqrt(D(2,2))*3; % 3-sigma semi-minor axis
304 angle = atan2(V(2,1), V(1,1));
305

```

```

306 ellipse_t = linspace(0, 2*pi, 100);
307 ellipse_x = x_x_est(1) + a_sigma*cos(ellipse_t)*cos(angle) - ...
308 b_sigma*sin(ellipse_t)*sin(angle);
309 ellipse_y = y_y_est(1) + a_sigma*cos(ellipse_t)*sin(angle) + ...
310 b_sigma*sin(ellipse_t)*cos(angle);
311 ellipse_h = plot(ellipse_x, ellipse_y, 'Color', 'r', 'LineWidth', 2);
312 % addpoints(h_ellipse, ellipse_x(1), ellipse_y(1));
313
314 %% Optional
315 % Initialize a placeholder for the lidar position plot
316 h_lidar = plot(0, 0, 'mo', 'MarkerSize', 6, 'LineWidth', 2); % 'm' for magenta color
317
318 % Initialize lines for range to landmarks
319 h_landmark_lines = gobjects(size(landmarks, 1), 1); % Placeholder for line handles
320 for l = 1:size(landmarks, 1)
321     h_landmark_lines(l) = line([0, 0], [0, 0], 'Color', 'g', 'LineWidth', 2);
322 end
323
324 %%
325 % Set the axis limits
326 axis equal;
327 xlim([min(x_true) - 1, max(x_true) + 1]);
328 ylim([min(y_true) - 1, max(y_true) + 1]);
329 legend('Landmarks', 'True Position', 'Estimated Position', '3-sigma Covariance
    Ellipse');
330 title('EKF Robot Localization');
331
332 % Animation loop
333 for k = 2:length(t)
334     % Update true and estimated robot positions
335     h_true.XData = x_true(k);
336     h_true.YData = y_true(k);
337
338     h_est.XData = x_x_est(k);
339     h_est.YData = y_y_est(k);
340
341     %% Optional
342     % h_open_loop.XData = x_open_loop(1, k);
343     % h_open_loop.YData = x_open_loop(2, k);
344     %%
345     % Update paths using animatedline
346     addpoints(h_true_path, x_true(k), y_true(k));
347     addpoints(h_est_path, x_x_est(k), y_y_est(k));
348
349     %% Optional
350     % addpoints(h_open_loop_path, x_open_loop(1, k), x_open_loop(2, k));
351     %%
352     % Update 3-sigma covariance ellipse
353     P_2x2 = P_posterior(1:2, 1:2, k);
354     [V, D] = eig(P_2x2);
355     a_sigma = sqrt(D(1,1))*3; % 3-sigma semi-major axis
356     b_sigma = sqrt(D(2,2))*3; % 3-sigma semi-minor axis
357     angle = atan2(V(2,1), V(1,1));
358     ellipse_x = x_x_est(k) + a_sigma*cos(ellipse_t)*cos(angle) - ...
359     b_sigma*sin(ellipse_t)*sin(angle);
360     ellipse_y = y_y_est(k) + a_sigma*cos(ellipse_t)*sin(angle) + ...
361     b_sigma*sin(ellipse_t)*cos(angle);
362     for n = 1:100
363         addpoints(h_ellipse, ellipse_x(n), ellipse_y(n));
364     end
365     ellipse_h.XData = ellipse_x;
366     ellipse_h.YData = ellipse_y;
367
368     %% Optional

```

```

367 %      % Compute the position of the lidar based on the robot's estimated position and
      orientation
368 lidar_x = x_x_est(k) + d * cos(x_posterior(3, k));
369 lidar_y = y_y_est(k) + d * sin(x_posterior(3, k));
370
371 % Update the lidar's position plot
372 h_lidar.XData = lidar_x;
373 h_lidar.YData = lidar_y;
374
375 % Plot range to landmarks
376 % Plot range to landmarks using estimated bearing and range data
377 for l = 1:size(landmarks, 1)
378     if r(k, l) > 0 && r(k, l) < rmax
379         end_x = lidar_x + r(k, l) * cos(b(k, l) + x_posterior(3, k));
380         end_y = lidar_y + r(k, l) * sin(b(k, l) + x_posterior(3, k));
381         h_landmark_lines(l).XData = [lidar_x, end_x];
382         h_landmark_lines(l).YData = [lidar_y, end_y];
383     else
384         h_landmark_lines(l).XData = [];
385         h_landmark_lines(l).YData = [];
386     end
387 end
388
389 %%
390 drawnow; % Refresh the plot
391
392 frame = getframe(fig); % fig gets the current figure handle
393 writeVideo(v, frame);
394
395
396 end
397
398 close(v);
399 hold off;
400
401 % Functions
402 Function to plot an ellipse
403 function plotEllipse(xCenter, yCenter, a, b, angle, varargin)
404     t = linspace(0, 2*pi, 100);
405     RotMatrix = [cos(angle) -sin(angle); sin(angle) cos(angle)];
406     xy = [a*cos(t); b*sin(t)];
407     xy = RotMatrix*xy;
408     plot(xy(1,:) + xCenter, xy(2,:) + yCenter, varargin{:});
409 end
410
411
412 function updateEllipse(h, x, y, P)
413     [V, D] = eig(P);
414     a = sqrt(D(1,1))*3; % 3-sigma semi-major axis
415     b = sqrt(D(2,2))*3; % 3-sigma semi-minor axis
416     angle = atan2(V(2,1), V(1,1));
417     t = linspace(0, 2*pi, 100);
418     ellipse_x = x + a*cos(t)*cos(angle) - b*sin(t)*sin(angle);
419     ellipse_y = y + a*cos(t)*sin(angle) + b*sin(t)*cos(angle);
420     hold on;
421     set(h, 'XData', ellipse_x, 'YData', ellipse_y);
422 end
423
424 function h = plotEllipse(x, y, P, varargin)
425     % PLOT ELLIPSE plots a 3-sigma covariance ellipse given a mean and covariance
426     % x, y: mean position
427     % P: 2x2 covariance matrix

```

```

428     % varargin: additional plotting parameters, e.g., 'Color', 'LineWidth'
429
430     [V, D] = eig(P);
431     a = sqrt(D(1,1))*3; % 3-sigma semi-major axis
432     b = sqrt(D(2,2))*3; % 3-sigma semi-minor axis
433     angle = atan2(V(2,1), V(1,1));
434     t = linspace(0, 2*pi, 100);
435     ellipse_x = x + a*cos(t)*cos(angle) - b*sin(t)*sin(angle);
436     ellipse_y = y + a*cos(t)*sin(angle) + b*sin(t)*cos(angle);
437     hold on;
438     h = plot(ellipse_x, ellipse_y, varargin{:});
439 end
440
441 function theta_wrapped = wrapToPi(theta)
442     % Wrap angle in radians to [-pi, pi]
443
444     theta_wrapped = mod(theta + pi, 2*pi) - pi;
445
446     % Handle the edge case where the result is -pi
447     % by setting it to pi
448     theta_wrapped(theta_wrapped == -pi) = pi;
449 end

```